

函数

本章学习目标

- 理解函数的概念和作用
 - 掌握函数的定义和调用方法
 - 熟练使用函数参数和返回值
 - 理解变量的作用域（全局变量与局部变量）
 - 掌握递归函数的编写技巧
 - 学会创建和使用函数库
 - 能够进行函数的调试和测试
-

7.1 函数基础

函数是一段可重复使用的代码块，它将一组命令组织在一起，通过名称调用执行。使用函数可以提高代码的可读性、可维护性和复用性。

7.1.1 为什么使用函数

使用函数的主要优势包括代码复用（相同的逻辑只需编写一次）、模块化（将复杂问题分解为小的可管理单元）、可维护性（修改只需在一处进行）、可读性（函数名可以描述其功能）以及可测试性（可以单独测试每个函数）。

7.1.2 函数定义语法

Shell 中定义函数有两种语法格式：

```
# 语法格式1（推荐）
function_name() {
    commands
}

# 语法格式2（使用 function 关键字）
```

```
function function_name {  
    commands  
}  
  
# 语法格式3 (结合两种方式)  
function function_name() {  
    commands  
}
```

【实操练习 7-1】函数基础示例

```
#!/bin/bash  
# 文件名: function_basic.sh  
  
# 定义一个简单的函数  
say_hello() {  
    echo "Hello, World!"  
}  
  
# 使用 function 关键字定义  
function say_goodbye {  
    echo "Goodbye!"  
}  
  
# 带有多条命令的函数  
show_system_info() {  
    echo "=== 系统信息 ==="  
    echo "主机名: $(hostname)"  
    echo "当前用户: $(whoami)"  
    echo "当前目录: $(pwd)"  
    echo "当前时间: $(date)"  
}  
  
# 调用函数  
echo "--- 调用 say_hello ---"  
say_hello  
  
echo -e "\n--- 调用 say_goodbye ---"  
say_goodbye  
  
echo -e "\n--- 调用 show_system_info ---"
```

```
show_system_info
```

运行结果：

```
--- 调用 say_hello ---  
Hello, World!  
  
--- 调用 say_goodbye ---  
Goodbye!  
  
--- 调用 show_system_info ---  
=== 系统信息 ===  
主机名: my-server  
当前用户: root  
当前目录: /root  
当前时间: Thu Dec 26 10:30:00 CST 2025
```



提示：函数必须先定义后调用。Shell 脚本是从上到下顺序执行的，如果在函数定义之前调用它，会报错 "command not found"。

7.2 函数参数

函数可以接收参数，参数通过位置变量 `$1`、`$2`、`$3` 等访问，与脚本接收命令行参数的方式相同。

7.2.1 传递和访问参数

```
#!/bin/bash  
# 文件名: function_params.sh  
  
# 定义带参数的函数  
greet() {  
    local name=$1  
    local greeting=${2:-"Hello"} # 默认值  
    echo "$greeting, $name!"  
}
```

```
# 显示参数信息
show_params() {
    echo "函数名: ${FUNCNAME[0]}"
    echo "参数个数: $# "
    echo "所有参数: $@"
    echo "第一个参数: $1"
    echo "第二个参数: $2"
    echo "第三个参数: $3"
}

# 调用带参数的函数
echo "=== greet 函数 ==="
greet "Alice"
greet "Bob" "Hi"
greet "Charlie" "Good morning"

echo -e "\n=== show_params 函数 ==="
show_params apple banana cherry date
```

运行结果:

```
=== greet 函数 ===
Hello, Alice!
Hi, Bob!
Good morning, Charlie!

=== show_params 函数 ===
函数名: show_params
参数个数: 4
所有参数: apple banana cherry date
第一个参数: apple
第二个参数: banana
第三个参数: cherry
```

7.2.2 函数内的特殊变量

变量	说明
<code>\$0</code>	脚本名称（不是函数名）
<code>\$1</code> – <code>\$9</code>	第1到第9个参数
<code>\${10}</code>	第10个及以后的参数
<code>\$#</code>	参数个数
<code>\$@</code>	所有参数（作为独立的字符串）
<code>\$*</code>	所有参数（作为单个字符串）
<code>\${FUNCNAME[0]}</code>	当前函数名
<code>\${FUNCNAME[1]}</code>	调用当前函数的函数名

【实操练习 7-2】参数处理进阶

```
#!/bin/bash
# 文件名: function_params_advanced.sh

# 处理可变数量的参数
sum_all() {
    local total=0
    for num in "$@"; do
        ((total += num))
    done
    echo $total
}

# 使用 shift 处理参数
process_files() {
    local action=$1
    shift # 移除第一个参数

    echo "操作: $action"
    echo "文件列表:"
    for file in "$@"; do
```

```

        echo "    - $file"
    done
}

# 参数验证
divide() {
    if [ $# -ne 2 ]; then
        echo "错误：需要两个参数" >&2
        return 1
    fi

    local dividend=$1
    local divisor=$2

    if [ "$divisor" -eq 0 ] 2>/dev/null; then
        echo "错误：除数不能为零" >&2
        return 2
    fi

    echo "$((dividend / divisor))"
}

# 测试 sum_all
echo "=== 求和函数 ==="
result=$(sum_all 1 2 3 4 5)
echo "1+2+3+4+5 = $result"

result=$(sum_all 10 20 30)
echo "10+20+30 = $result"

# 测试 process_files
echo -e "\n=== 处理文件 ==="
process_files "backup" file1.txt file2.txt file3.txt

# 测试 divide
echo -e "\n=== 除法函数 ==="
divide 10 2
divide 10 0
divide 10

```

7.3 函数返回值

Shell 函数有两种方式返回结果：使用 `return` 语句返回退出状态码，或使用 `echo` 输出结果。

7.3.1 return 语句

`return` 语句用于返回一个 0–255 之间的整数作为函数的退出状态码。按照惯例，0 表示成功，非零表示失败或特定的错误代码。

```
#!/bin/bash
# 文件名: function_return.sh

# 使用 return 返回状态码
is_file_exists() {
    local filename=$1
    if [ -f "$filename" ]; then
        return 0 # 成功（文件存在）
    else
        return 1 # 失败（文件不存在）
    fi
}

# 检查数字是否为偶数
is_even() {
    local num=$1
    if (( num % 2 == 0 )); then
        return 0 # 是偶数
    else
        return 1 # 不是偶数
    fi
}

# 使用函数的返回值
echo "=== 检查文件 ==="
if is_file_exists "/etc/passwd"; then
    echo "/etc/passwd 存在"
else
    echo "/etc/passwd 不存在"
fi
```

```

if is_file_exists "/nonexistent"; then
    echo "/nonexistent 存在"
else
    echo "/nonexistent 不存在"
fi

# 使用 && 和 || 处理返回值
echo -e "\n=== 检查偶数 ==="
for num in 2 5 8 11; do
    is_even $num && echo "$num 是偶数" || echo "$num 是奇数"
done

# 获取返回值
echo -e "\n=== 获取返回值 ==="
is_even 10
echo "返回值: $?"

is_even 7
echo "返回值: $?"

```

7.3.2 使用 echo 返回结果

当需要返回字符串或数值结果时，使用 `echo` 输出，然后通过命令替换捕获：

```

#!/bin/bash
# 文件名: function_echo_return.sh

# 返回计算结果
calculate_area() {
    local length=$1
    local width=$2
    local area=$((length * width))
    echo $area # 输出结果
}

# 返回字符串
get_username() {
    local uid=$1
    local username=$(getent passwd "$uid" | cut -d: -f1)
}

```



```

    echo "${username:-unknown}"
}

# 返回多个值（使用特定分隔符）
get_file_info() {
    local file=$1
    if [ -f "$file" ]; then
        local size=$(stat -c %s "$file" 2>/dev/null)
        local mtime=$(stat -c %Y "$file" 2>/dev/null)
        local perms=$(stat -c %a "$file" 2>/dev/null)
        echo "${size}:${mtime}:${perms}" # 用冒号分隔
    else
        echo "0:0:000"
    fi
}

# 使用函数返回值
echo "=== 计算面积 ==="
area=$(calculate_area 5 3)
echo "长5宽3的面积: $area"

echo -e "\n=== 获取用户名 ==="
user=$(get_username 0)
echo "UID 0 的用户名: $user"

user=$(get_username 1000)
echo "UID 1000 的用户名: $user"

echo -e "\n=== 获取文件信息 ==="
info=$(get_file_info "/etc/passwd")
echo "原始信息: $info"

# 解析返回的多个值
IFS=':' read -r size mtime perms <<< "$info"
echo "大小: $size 字节"
echo "修改时间: $(date -d @$mtime 2>/dev/null || echo $mtime)"
echo "权限: $perms"

```

【图7-1：函数返回值方式对比】

函数返回值的两种方式

return 语句

echo 输出

返回范围：0-255 整数

返回范围：任意字符串或数值

获取方式：\$?

获取方式：\$(function_name)

用途：表示成功/失败状态

用途：返回计算结果或数据

示例：

示例：

is_valid() {

get_sum() {

[-f "\$1"]

echo \$((\$1 + \$2))

return \$?

}

}

result=\$(get_sum 3 5)

if is_valid "file"; then

echo "结果: \$result"

...

fi

7.4 变量作用域

理解变量的作用域对于编写正确的函数至关重要。

7.4.1 全局变量与局部变量

```
#!/bin/bash
# 文件名: variable_scope.sh

# 全局变量
global_var="我是全局变量"
counter=0

# 函数中的变量作用域
demo_scope() {
    # 未使用 local 的变量是全局的
    global_in_func="在函数中定义的全局变量"

    # 使用 local 定义局部变量
    local local_var="我是局部变量"

    # 修改全局变量
    counter=100

    echo "函数内部:"
    echo "  global_var = $global_var"
    echo "  local_var = $local_var"
    echo "  counter = $counter"
}

echo "=== 调用函数前 ==="
echo "global_var = $global_var"
echo "counter = $counter"
echo "local_var = $local_var" # 空
echo "global_in_func = $global_in_func" # 空

echo -e "\n=== 调用函数 ==="
demo_scope

echo -e "\n=== 调用函数后 ==="
```

```
echo "global_var = $global_var"
echo "counter = $counter" # 被修改为 100
echo "local_var = $local_var" # 空 (局部变量已销毁)
echo "global_in_func = $global_in_func" # 有值
```

运行结果：

```
=== 调用函数前 ===
global_var = 我是全局变量
counter = 0
local_var =
global_in_func =

=== 调用函数 ===
函数内部：
    global_var = 我是全局变量
    local_var = 我是局部变量
    counter = 100

=== 调用函数后 ===
global_var = 我是全局变量
counter = 100
local_var =
global_in_func = 在函数中定义的全局变量
```

7.4.2 local 关键字详解

```
#!/bin/bash
# 文件名: local_keyword.sh

# 使用 local 的最佳实践
safe_function() {
    # 始终使用 local 声明函数内的变量
    local input=$1
    local result=""
    local i

    for i in $(seq 1 $input); do
        result="${result}*"
    done
}
```

```

done

echo "$result"
}

# 局部变量可以与全局变量同名
name="Global Name"

test_shadowing() {
    local name="Local Name" # 遮蔽全局变量
    echo "函数内 name = $name"
}

echo "=== safe_function ==="
output=$(safe_function 5)
echo "输出: $output"
echo "i 的值: ${i:-未定义}" # i 是局部的，不存在

echo -e "\n=== 变量遮蔽 ==="
echo "调用前 name = $name"
test_shadowing
echo "调用后 name = $name" # 全局变量未受影响

```

7.9 本章小结

本章我们全面学习了 Shell 函数的使用。

函数定义与调用方面，我们学习了两种定义语法 `name() {}` 和 `function name {}`，函数必须先定义后调用，函数名遵循变量命名规则。

参数传递方面，我们了解到函数使用 `$1`、`$2` 等位置参数，`$#` 表示参数个数，`$@` 和 `$*` 表示所有参数，`shift` 用于移动参数位置。

返回值方面，`return` 返回 0-255 的退出状态码，`echo` 输出结果，用命令替换捕获，可以结合使用状态码和输出。

变量作用域方面，默认是全局变量，`local` 声明局部变量，局部变量在函数结束时销毁，嵌套函数可以访问外层的局部变量。

函数库方面，使用 `source` 或 `.` 加载函数库，合理组织函数库结构，添加必要的文档注释。